
Evaluación Final Front-End

Documentación sobre el proceso de desarrollo

NOMBRE: Don Enrique Eduardo Alegría Saphier
CARRERA: Analista de datos e Ingeniero Informático
ASIGNATURA: Programación Front-End - TI3031
PROFESOR: Vedran Hrvoj Tomicic Cantizano
FECHA: 11-07-2026

1 Introducción

Los laboratorios de una institución educativa suelen administrar sus recursos tecnológicos —notebooks, routers, switches, access points, impresoras 3D, sensores IoT, kits Arduino, cámaras y herramientas de red, entre otros— mediante registros manuales en planillas o cuadernos. Esta forma de trabajo genera pérdida de información, duplicidad de datos y falta de claridad respecto del estado y la disponibilidad real de cada recurso.

Para responder a esta necesidad se desarrolló “Gestión de Recursos Lab”, una aplicación web de tipo SPA (Single Page Application) construida con React y Next.js, que permite registrar, listar, editar, eliminar, buscar y filtrar los recursos tecnológicos de un laboratorio directamente desde el navegador. La solución utiliza los tres mecanismos de almacenamiento del navegador solicitados en la evaluación: Local Storage para persistir el listado principal de recursos, Session Storage para mantener los filtros y la búsqueda mientras dura la sesión, y Cookies para guardar preferencias simples del usuario, como el tema visual (claro/oscuro) y su nombre visible.

Este documento describe el proceso de desarrollo seguido para construir la aplicación: la instalación y configuración inicial del proyecto, la definición de tipos de datos y validaciones, la construcción de los hooks personalizados, el uso de los mecanismos de almacenamiento del navegador, los componentes principales de la interfaz, el apoyo de inteligencia artificial durante el desarrollo, la evidencia visual del funcionamiento del sistema y las conclusiones del equipo.

2 Objetivo General y Tecnologías Utilizadas

El objetivo general de este proyecto es desarrollar una aplicación web SPA con React y Next.js que permita gestionar los recursos tecnológicos de un laboratorio mediante operaciones CRUD completas, utilizando Local Storage como almacenamiento principal de los datos, Session Storage para la información temporal de sesión y Cookies para las preferencias simples del usuario.

Para su construcción se utilizaron las siguientes tecnologías:

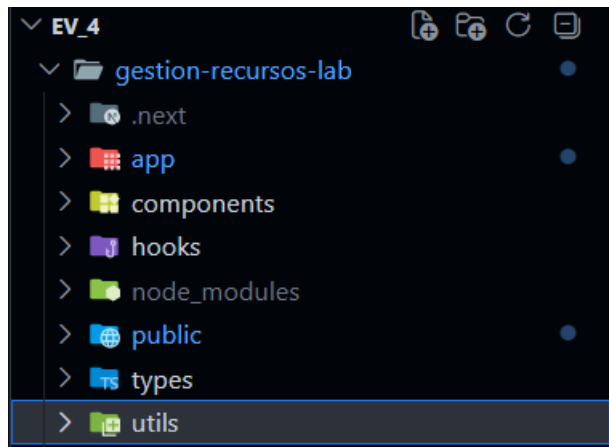
- Next.js 16 (App Router), como framework principal de la aplicación.
- React 19, para la construcción de la interfaz mediante componentes y hooks.
- TypeScript, para tipar los datos de los recursos y reducir errores en tiempo de desarrollo.
- Tailwind CSS 4, para el diseño y los estilos de la interfaz, incluyendo el modo claro/oscuro.
- APIs nativas del navegador: localStorage, sessionStorage y document.cookie.

3 Instalación y Estructura Inicial del Proyecto

Procederemos a instalar primero el marco de trabajo para “gestión-recursos-lab”

```
C:\Users\MrRaz\Desktop\ev_final_Front-end\ev_4>npx create-next-app@latest gestion-recursos-lab --typescript --eslint --tailwind --app
```

Nos aseguramos de crear la estructura que vamos a utilizar



Solo tuvimos que crear

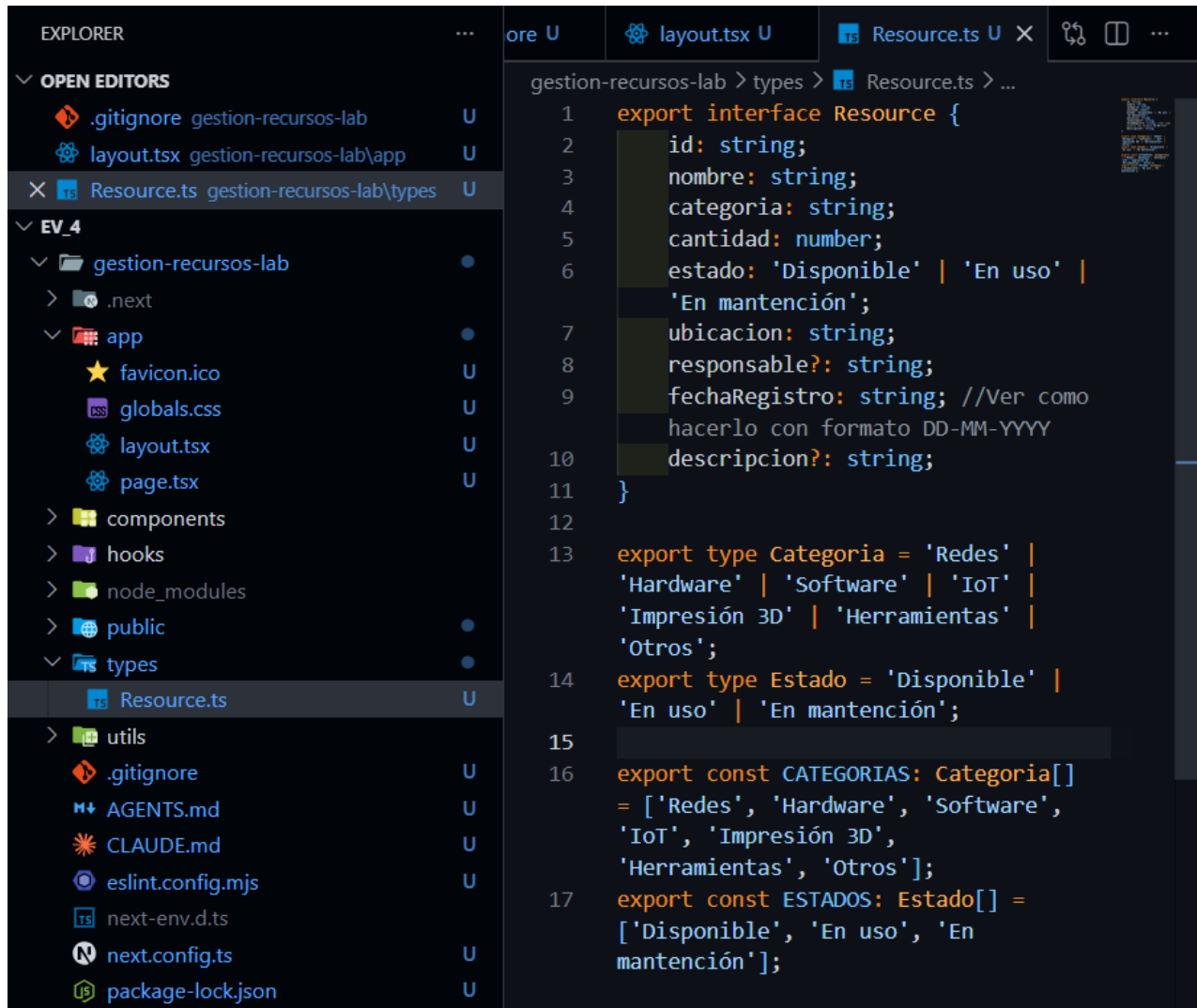
Components

Hooks

Types

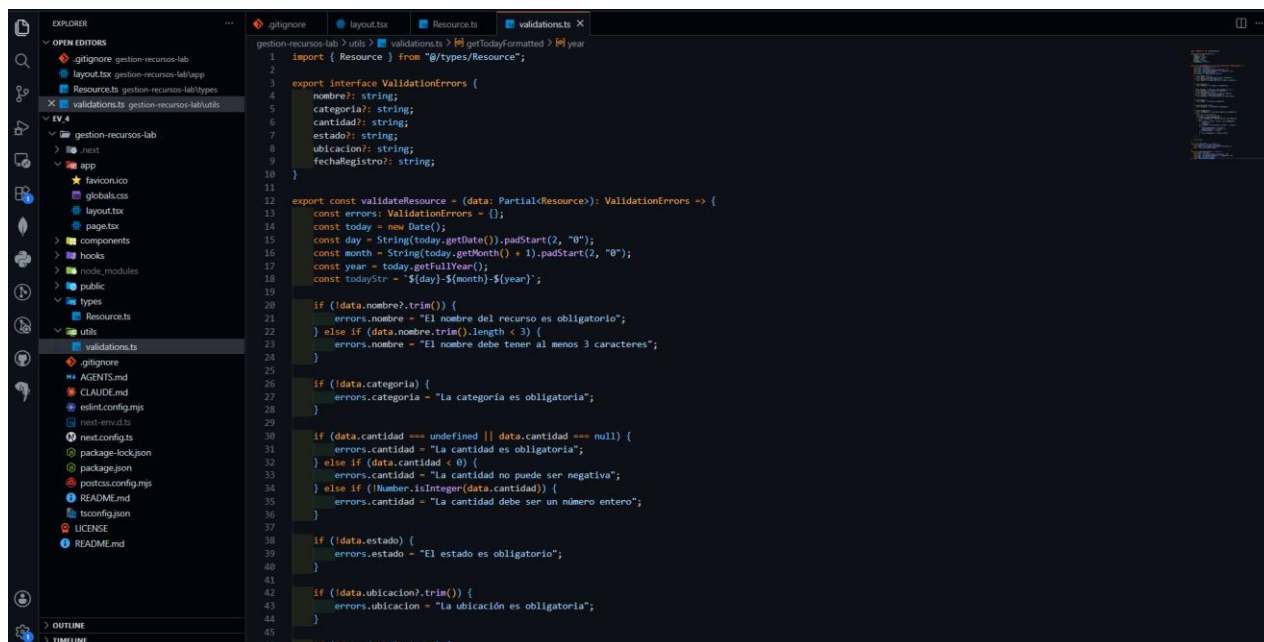
Utils

Lo primero es en types creamos Resource.ts para dar formato a los datos que se solicitaran



```
1  export interface Resource {
2      id: string;
3      nombre: string;
4      categoria: string;
5      cantidad: number;
6      estado: 'Disponible' | 'En uso' |
7          'En mantención';
8      ubicacion: string;
9      responsable?: string;
10     fechaRegistro: string; //Ver como
11     hacerlo con formato DD-MM-YYYY
12     descripcion?: string;
13 }
14
15 export type Categoria = 'Redes' |
16     'Hardware' | 'Software' | 'IoT' |
17     'Impresión 3D' | 'Herramientas' |
18     'Otros';
19
20 export type Estado = 'Disponible' |
21     'En uso' | 'En mantención';
22
23 export const CATEGORIAS: Categoria[] =
24     ['Redes', 'Hardware', 'Software',
25     'IoT', 'Impresión 3D',
26     'Herramientas', 'Otros'];
27
28 export const ESTADOS: Estado[] =
29     ['Disponible', 'En uso', 'En
30     mantención'];
```

También creamos definiciones para Categorías , establecemos Estado, etc



Procedemos a crear las validaciones.

3.1 Tipos de Datos y Validaciones

En types/Resource.ts se definió la interfaz Resource, que establece la forma de los datos de cada recurso tecnológico: id, nombre, categoría, cantidad, estado, ubicación, responsable (opcional), fecha de registro y descripción (opcional). También se definieron los tipos Categoría y Estado junto con sus listas de valores permitidos (CATEGORIAS y ESTADOS), que se reutilizan en el formulario y en los filtros de la aplicación.

Junto con los tipos, en utils/validations.ts se implementó la función validateResource, encargada de validar el formulario antes de guardar un recurso. Entre las reglas aplicadas se encuentran:

- El nombre del recurso es obligatorio y debe tener al menos 3 caracteres.
- La categoría, el estado y la ubicación son campos obligatorios.
- La cantidad es obligatoria, debe ser un número entero y no puede ser negativa.
- La fecha de registro es obligatoria; se valida tanto el formato (DD-MM-AAAA) como que corresponda a una fecha calendario real.
- El responsable y la descripción son campos opcionales.
- Si existen errores, estos se muestran junto a cada campo del formulario y se impide guardar un recurso con datos incompletos o inválidos.

3.2 Hooks Personalizados

Para conectar la interfaz con el almacenamiento del navegador se crearon tres hooks personalizados dentro de la carpeta hooks/, cada uno encapsulando la lectura y escritura de un mecanismo distinto. Los tres siguen el mismo patrón: leen el valor guardado al montar el componente mediante useEffect, mantienen el valor actual en un estado con useState, y exponen una función setValue que actualiza tanto el estado como el almacenamiento correspondiente.

useLocalStorage(key, initialValue): se utiliza en la página principal para guardar y recuperar el arreglo completo de recursos (lab_resources), que es la fuente de verdad del CRUD.

useSessionStorage(key, initialValue): se utiliza para mantener el término de búsqueda y los filtros de categoría y estado (lab_search, lab_filter_category, lab_filter_status) mientras dura la sesión del navegador.

useCookie(key, initialValue, days): se utiliza en el encabezado de la aplicación para guardar el tema visual (theme) y el nombre visible del usuario (user_name), preferencias que se conservan por 30 días.

Todos los hooks, junto con los componentes que los utilizan (Header, ThemeToggle y la página principal), están marcados con la directiva “use client”, ya que acceden a APIs del navegador (window, document) que no existen durante el renderizado en el servidor.

4 Persistencia de Datos en el Navegador

La aplicación no utiliza un backend ni una base de datos: toda la información se guarda directamente en el navegador del usuario, distribuida en los tres mecanismos solicitados por la evaluación, cada uno con un propósito distinto:

Local Storage (lab_resources): almacena el listado completo de recursos tecnológicos. Es el mecanismo elegido porque los datos deben persistir aunque se cierre la pestaña o el navegador, ya que representan la información principal del sistema.

Session Storage (lab_search, lab_filter_category, lab_filter_status): almacena el término de búsqueda y los filtros activos. Al ser información temporal de la sesión actual, se limpia automáticamente al cerrar la pestaña, evitando que un filtro quede “pegado” en visitas futuras.

Cookies (theme, user_name): almacenan preferencias simples del usuario —el tema visual claro/oscuro y el nombre visible en el encabezado— con una expiración de 30 días. No se guarda en ningún momento información sensible, contraseñas ni datos personales críticos, conforme a las condiciones de la evaluación.

5 Componentes Principales

La interfaz se organizó en componentes reutilizables dentro de la carpeta components/, separando la presentación de la lógica de datos, que vive principalmente en app/page.tsx:

Header: muestra el título de la aplicación, el contador de recursos registrados, el campo de nombre de usuario (cookie) y el botón de cambio de tema.

ResourceForm: formulario para crear y editar recursos, con validación de campos en tiempo real.

ResourceList: muestra el listado de recursos filtrados, o un mensaje vacío cuando no hay resultados.

ResourceCard: representa cada recurso individual en formato de tarjeta, con su estado, categoría, ubicación y acciones de editar/eliminar.

SearchBar: permite buscar recursos por nombre.

FilterCategory: permite filtrar recursos por categoría y por estado.

ThemeToggle: cambia entre modo claro y oscuro, guardando la preferencia en una cookie.

ConfirmDeleteModal: solicita confirmación antes de eliminar un recurso, evitando borrados accidentales.

6 Uso de Inteligencia Artificial

Durante el desarrollo se utilizó inteligencia artificial como apoyo al proceso, principalmente en las siguientes tareas:

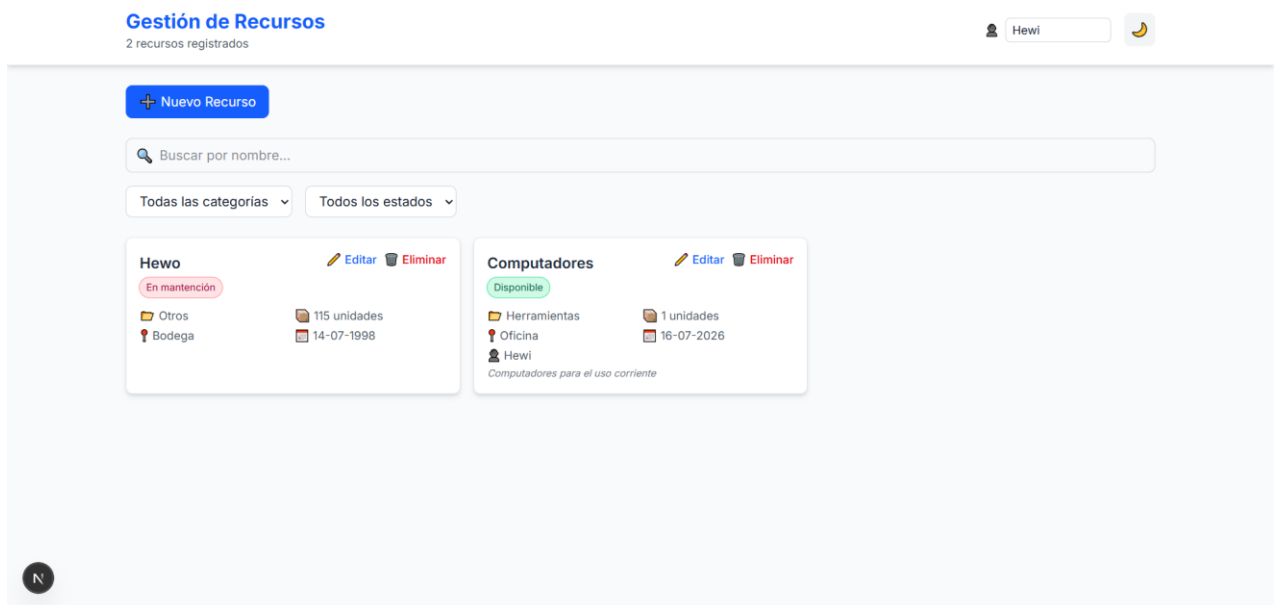
- Generación de una estructura inicial de componentes, hooks y tipos, a partir de la cual el equipo continuó desarrollando y ajustando el código.
- Sugerencias de validaciones para el formulario de recursos.
- Apoyo en la corrección de errores y en la explicación de mensajes de error durante el desarrollo.
- Recomendaciones de buenas prácticas para el uso de Local Storage, Session Storage y Cookies.

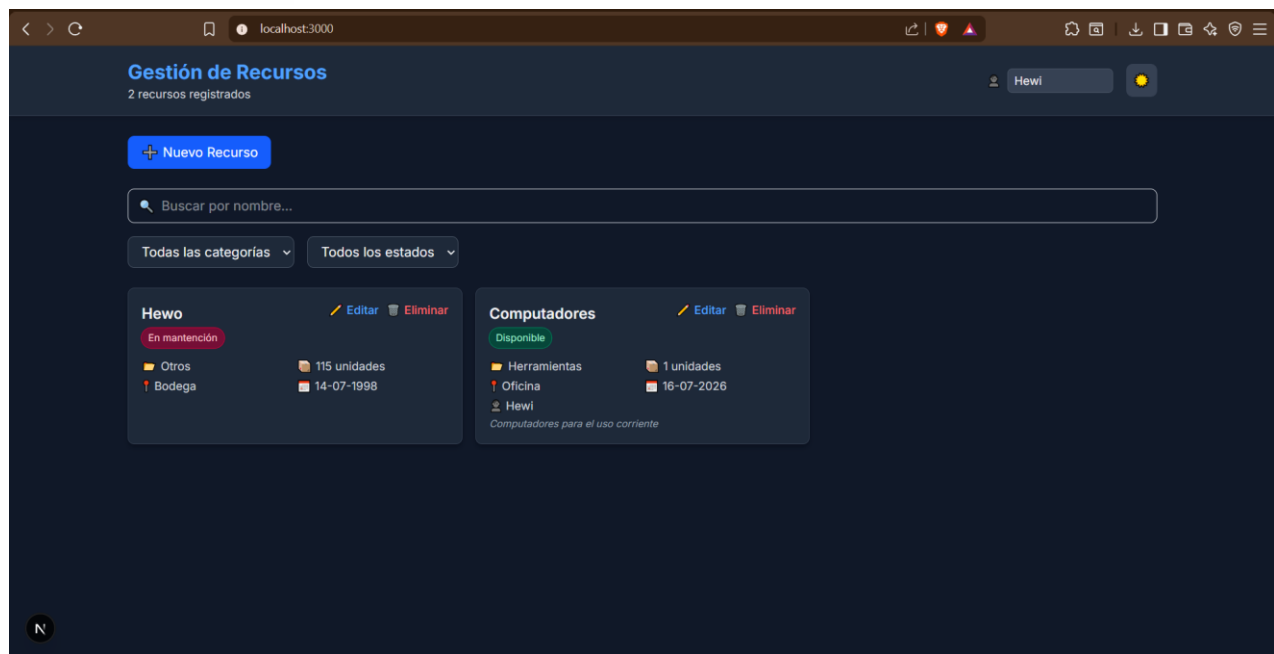
El equipo revisó, comprende y puede explicar el funcionamiento de cada parte del código entregado, tal como lo exige la evaluación; el uso de IA se limitó a un rol de apoyo y no reemplazó las decisiones técnicas del equipo.

7 Capturas de Pantalla del CRUD en Funcionamiento

A continuación se debe incluir evidencia visual de la aplicación en ejecución. Se recomienda agregar, como mínimo, capturas de:

- El listado de recursos con datos de ejemplo cargados.
- El formulario de creación de un recurso, incluyendo algún error de validación visible.
- La edición de un recurso existente.
- El modal de confirmación antes de eliminar un recurso.
- La búsqueda y los filtros por categoría/estado en uso.
- La aplicación en modo claro y en modo oscuro (Theme Toggle).





8 Conclusión

El desarrollo de este proyecto permitió aplicar de forma práctica los conceptos de componentes, hooks y almacenamiento del navegador en una aplicación React con Next.js. Construir los tres hooks personalizados (useLocalStorage, useSessionStorage y useCookie) ayudó a comprender en detalle las diferencias entre Local Storage, Session Storage y Cookies, y en qué casos conviene usar cada uno según la persistencia y el alcance que se necesite.

Como equipo, el mayor aprendizaje estuvo en separar correctamente la lógica de datos de la interfaz, validar la información antes de guardarla y organizar el proyecto en carpetas coherentes (components, hooks, types, utils), lo que facilitó el trabajo colaborativo y la mantención del código a lo largo del desarrollo.